

# Team Workflow

Branching, PR process, and the order we integrate in.

Read this if: everyone

## The team

M1 Team Lead/Backend/API/Integration · M2 Repository + Git History · M3 Static Code Intelligence + AI + Risk/Offboarding · M4 Frontend A (Shell + Workspace) · M5 Frontend B (Intelligence + Knowledge)

## Branches

main (stable, submission-ready) · develop (integration) · feature/backend-api · feature/repository-history · feature/ai-intelligence · feature/frontend-workspace · feature/frontend-intelligence

- Nobody pushes directly to main
- PRs go into develop ; feature branches are cut from develop
- Pull the latest develop before starting or resuming work
- Conflicts get resolved with the owner of the affected module, not around them

## Work sequence

1. **Freeze** — everyone reads the Build Spec, PRD, API Contract, Database Design, this document, and Team Assignments before writing code.
2. **Skeleton** — module skeletons and interfaces before deep implementation.
3. **Parallel build** — everyone builds within their ownership boundary.
4. **Early integration** — don't wait until the last hours: Repository → API, then Repository+History → AI, then Backend → Frontend.
5. **End-to-end** — test the full demo flow start to finish.

## Before opening a PR

Confirm it runs locally, no secrets are committed, tests cover the logic that matters, the API contract and database design are respected, there's no unrelated refactor riding along, and commit messages explain the change.

feature/\* → develop → main (only once stable and integrated)

## Using AI coding assistants

Assistants can write the code, but not make architecture calls. They shouldn't change the stack, add a database, alter API contracts, invent new agents, expand scope, execute repository code, or commit secrets. Reusable prompt:

Read MASTER\_BUILD\_SPEC.md, PRD.md, API\_CONTRACT.md and DATABASE\_DESIGN.md first.  
Implement only the assigned module.  
Do not change architecture, API contracts, database design, technology stack, agent responsibilities or scope.  
If you detect a conflict, stop and report it instead of changing the specification.

## Communication rule

Need something from another module? Ask for it — don't go modify their code directly. Example: M3 needs repo history → M2 exposes the agreed service → M3 integrates against that interface.

## Integration ownership

The Team Lead owns final integration. Code on GitHub isn't "integrated" until it's merged into develop , starts successfully, connects to its dependencies, passes its tests, and works in the end-to-end demo.